

Quantum Shield

Three-Tier Architecture Document

Quantum-Proof Systems Scanner - PNB Cybersecurity Hackathon

Team CyberRiot - July 18, 2026

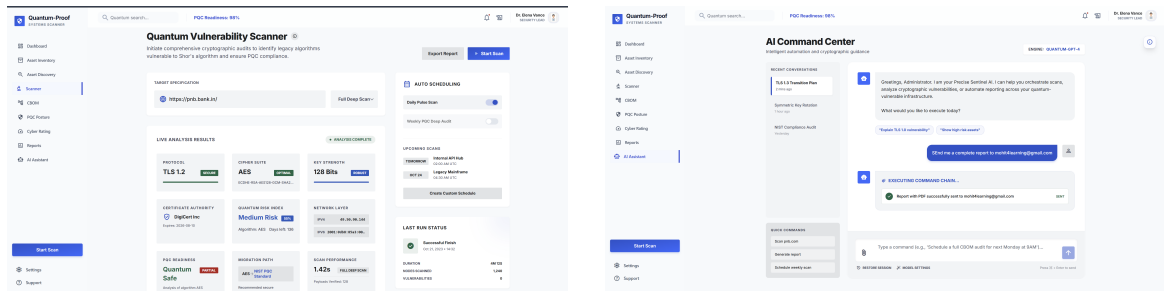
Contents

1 High-Level Overview.	1
2 Three-Tier Architecture Diagram.	2
3 Presentation Tier (Frontend).	2
4 Application Tier (Backend).	3
5 Data Tier.	4
6 PNB Hackathon Requirement Mapping.	4
7 Deployment and Runtime Flow.	5

Repository: https://github.com/Mohitlikestocode/Quantum-Proof-Systems-Scanner_PNB

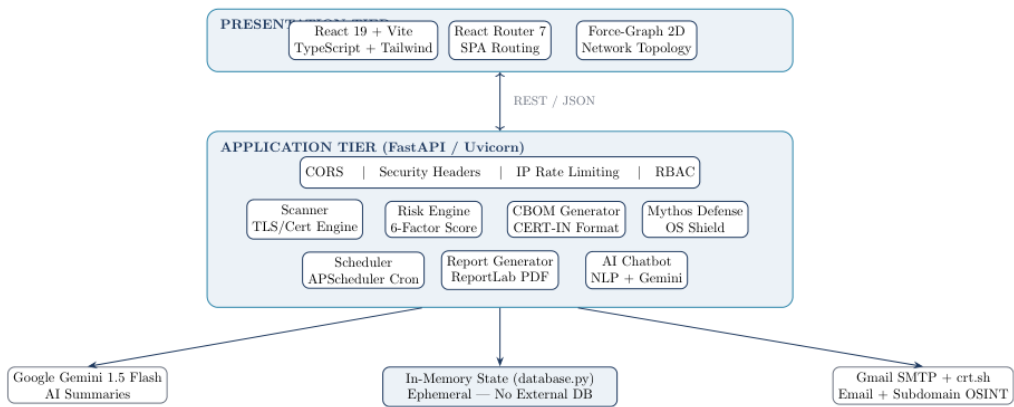
1. High-Level Overview

Quantum Shield is a decoupled web platform designed to evaluate enterprise cryptographic posture against quantum-era risks. The system separates concerns into Presentation, Application, and Data tiers so that scanning logic, risk analytics, and reporting workflows remain modular, testable, and extensible. The architecture supports full domain and subdomain scanning, TLS and certificate intelligence, vulnerability checks, PQC readiness scoring, executive PDF exports, role-aware controls, and AI-assisted automation for scheduling and reporting.

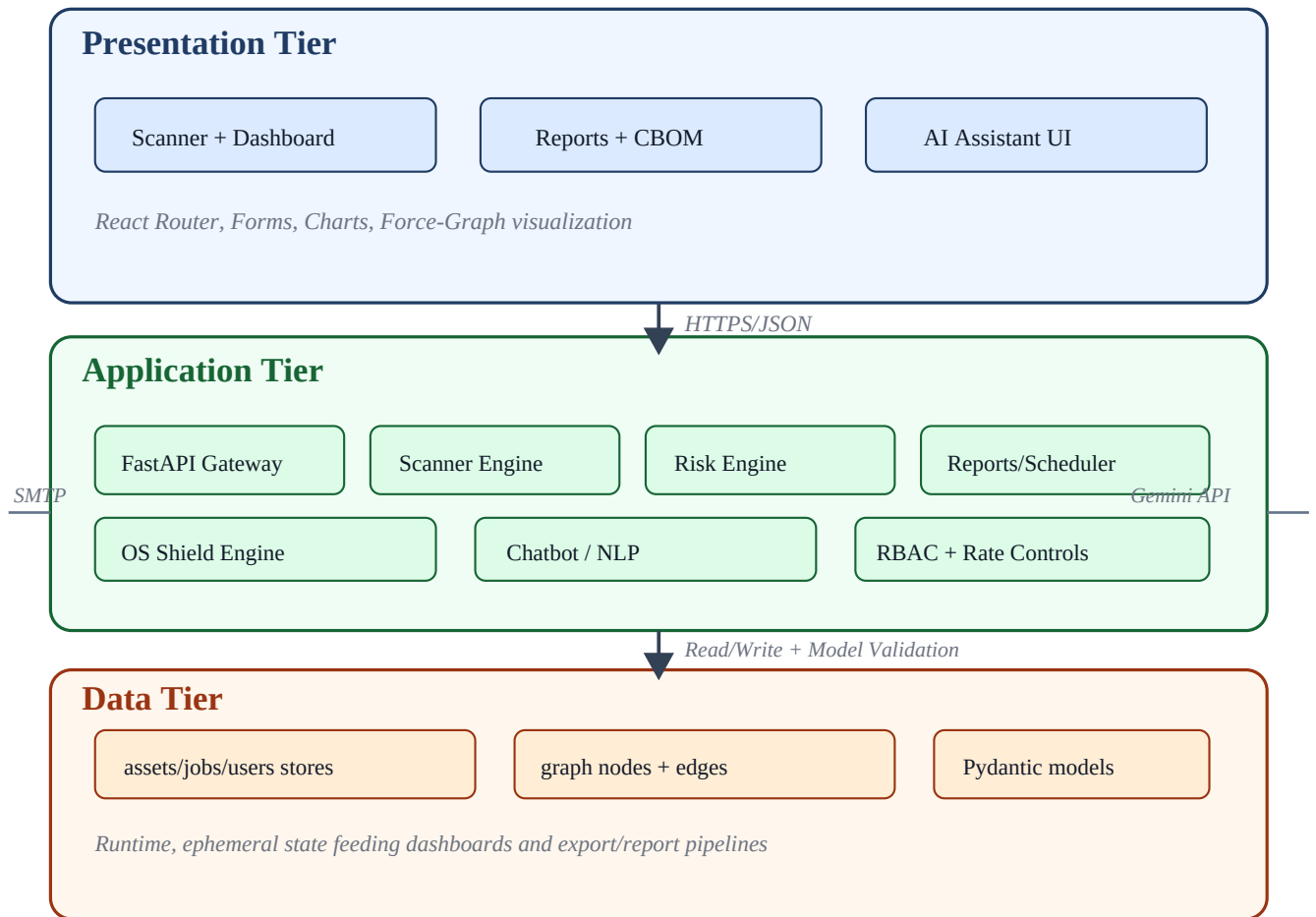


2. Three-Tier Architecture Diagram

Reference System Diagram (from the original system_architecture.pdf)



Three-Tier Logical Flow Diagram



Flow Execution Walkthrough

Step	Primary Tier	Execution Detail	Output Artifact
1	Presentation	User submits target and scan mode from Scanner/Dashboard modules.	Validated request payload
2	Application	FastAPI validates schema, applies governance checks, and routes to engines.	Engine execution context
3	Application	Scanner + risk engines generate protocol, vulnerability, and PQC posture metrics.	Risk-annotated scan record
4	Data	Runtime stores persist assets, jobs, graph topology, and user-scoped state.	Unified runtime state
5	Application	Report and scheduler modules convert state into PDFs, email tasks, and recurring jobs.	Executive reports + queued jobs
6	Presentation	Dashboard, reports, and AI assistant render findings and trigger follow-up actions.	Operator decisions + audit trail

Security and Reliability Controls

- RBAC checks are enforced before privileged reporting and user-management routes.
- Input schemas and request validation reduce malformed execution and workflow drift.
- Scheduler isolation keeps recurring jobs independent from immediate interactive scans.
- PDF export and email delivery pipelines are decoupled from UI rendering for resilience.

Operational Contract Matrix

Contract	Producer	Consumer	Validation Rule
Scan Request	Scanner UI	FastAPI /api/scan	Domain format + mode whitelist
Risk Payload	scanner.py	risk_engine.py	TLS fields and crypto metadata required
Topology Graph	Backend graph store	Dashboard force graph	nodes/edges schema integrity
Executive Report	report_generator.py	Reports module	asset summary + risk bands + timestamps
Scheduler Job	Scheduler UI / AI action	scheduler.py	interval + target + recipient checks

Operational Notes

This contract model ensures tier decoupling: frontend modules remain presentation-only, API endpoints own orchestration, and runtime stores stay authoritative for report and dashboard projections.

3. Presentation Tier (Frontend)

Stack

React, Vite, TypeScript, Tailwind CSS, React Router, Force Graph visualization.

Responsibilities

- User authentication flow and role context handling.
- Scan controls (target, mode, scheduler) and live telemetry display.
- Dashboard KPIs, heatmaps, graph views, and report interaction.
- AI assistant interface for natural-language operations.

Primary UI Modules

Module	Purpose
Dashboard	Aggregate risk posture, heatmap, and quick exports
Scanner	Domain/subdomain scan execution and detailed findings
Reports	JSON/PDF exports, history, and company report email workflows
Mythos Defense	OS vulnerability and patch-deficit simulation with export artifacts
AI Assistant	Action parser interactions for schedule, email, and scan automation

4. Application Tier (Backend)

Core API and Engine Roles

Layer	Implementation	Role
API Gateway	FastAPI main.py	Validation, rate limiting, endpoint orchestration, RBAC gatekeeping
Scanner Engine	scanner.py	TLS/certificate checks, subdomain discovery, vulnerability probes, mobile signals
Risk Engine	risk_engine.py	Weighted quantum-risk scoring, dual TLS compatibility penalty, classification
Report Engine	report_generator.py	Executive-grade PDF rendering for modular and website reports
Scheduler	scheduler.py	Recurring scan jobs, trigger logic, and automated dispatch workflows
AI/Email	chatbot.py	Intent parsing, optional Gemini summarization, SMTP report delivery
OS Shield	os_shield_engine.py	Windows vulnerability mapping, migration profile, and bundle generation

5. Data Tier

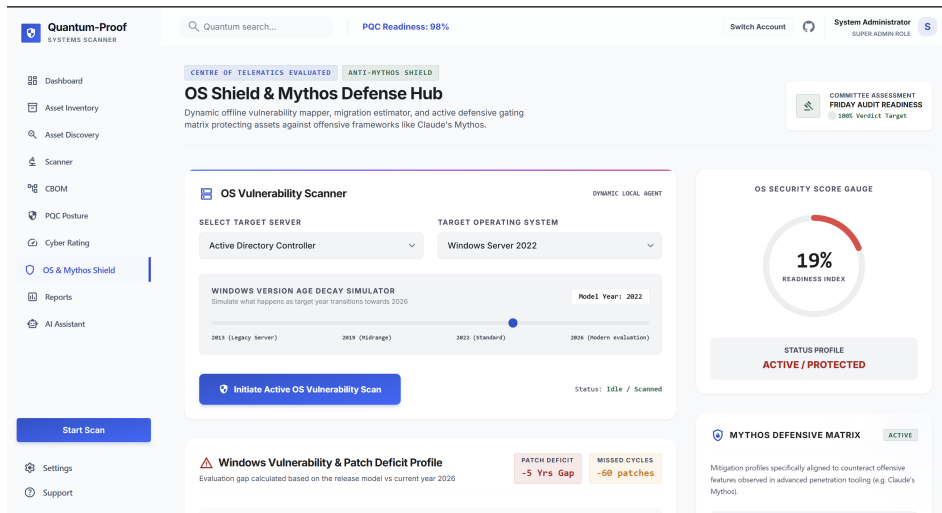
Data is maintained in runtime in-memory stores for assets, jobs, users, topology nodes, and edges. Pydantic schemas enforce structured exchange between endpoint handlers and UI consumers.

Data Domain	Storage Construct	Examples
Asset Inventory	db_assets dict	scan_result, risk profile, vulnerabilities, hosting, mobile applications
Scheduling	db_jobs list	recurring scan metadata, cadence config, and trigger context
Topology	db_nodes/db_edges lists	graph rendering payloads for infrastructure relationships
Identity	db_users dict + OTP store	role-bound sessions, login verification, and governance checks

6. PNB Hackathon Requirement Mapping

PNB Requirement Focus	Architecture Response
Full domain + subdomain scan	Scanner engine discovers and classifies active/inactive subdomains using multiple discovery sources
Mathematical risk model	Dedicated weighted risk engine with protocol, crypto, vulnerability, exposure, and governance factors
Report segmentation	Asset discovery, subdomain risk, vulnerability, and mobile report APIs + downloadable PDFs
Role control and governance	API-level role checks for report export and user management operations
Mobile app discovery	Brand-aware Android and iOS app matching integrated in the scan pipeline
Vulnerability + hosting intelligence	HTTP checks plus hosting inference and severity summary aggregation
OS hardening and migration	Mythos OS module with patch deficits, migration estimation, PDF, and ZIP bundle
Automation	Scheduler and AI action parsing for recurring scans and email dispatch

Mythos Defensive Module Snapshot



7. Deployment and Runtime Flow

1. Frontend route issues scan/report/auth request over HTTP.
2. API layer validates domain/email payloads and applies rate limits.
3. Scanner and risk engines process discovery, classification, and scoring.
4. Data tier stores runtime state for dashboards, exports, and history.
5. Report/email/scheduler modules produce executive outputs and automation.

This document is tailored for PNB Hackathon architecture evaluation and aligns directly with implemented modules.